OnTrack — Build Plan

Oct 28, 2026 · Katie

How to read this

This is the build order for OnTrack: nine phases, each one a working slice of the app that can be tried on a phone before the next starts. The Stack & Build Plan from October 20 holds the pre-design exploration (why a PWA, why Supabase, the auth alternatives, storage costs); this doc supersedes its phases and stack table.

The goal of the first four phases is one thing: an app the Four Corners ops team can install on their phones and use for real tickets, because a working app will land better than the slides and the design canvas have. Everything in the design (about 110 screens) is covered by a phase here; nothing is dropped, only ordered.

Each phase lists what it ships, the screens it covers from the design, the backend it needs, and a done test. Screens are named as they are on the design board so you can open them side by side.

Decisions

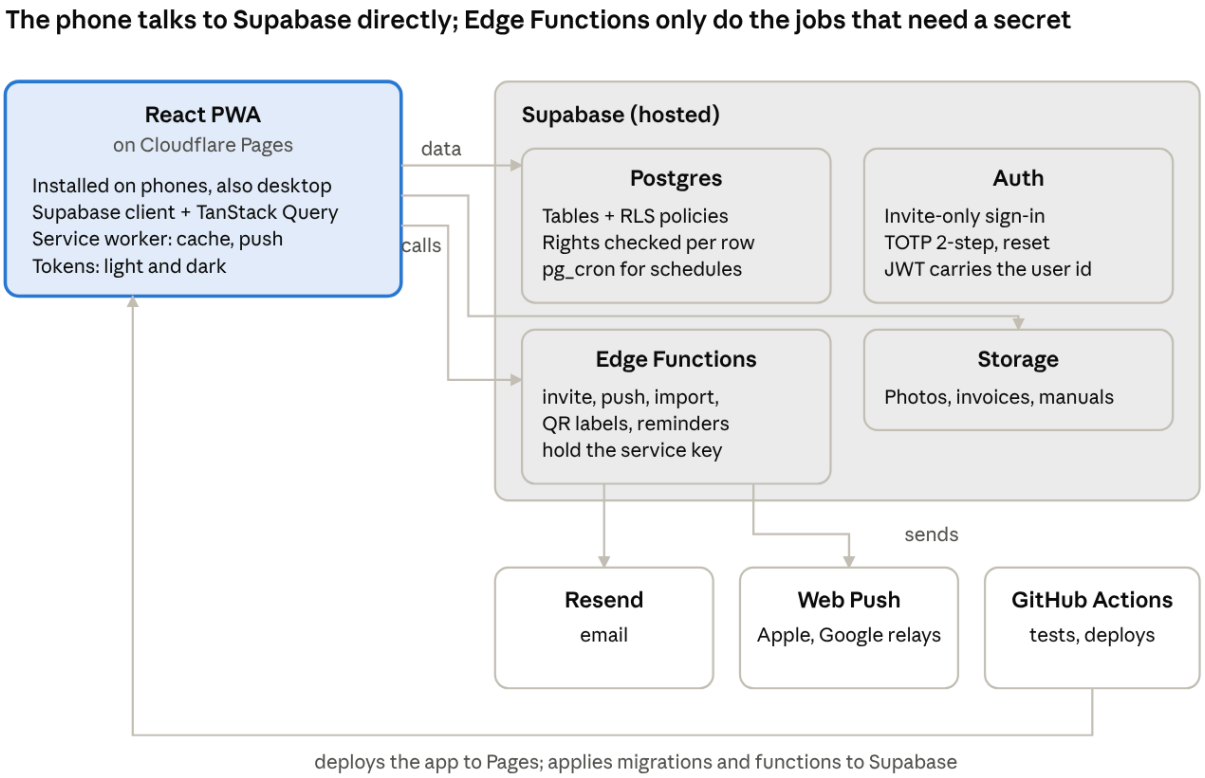
The app talks to Supabase directly and Edge Functions handle only the jobs that need a secret or a server. That is the smallest backend that still keeps roles enforced in the database.

Area	Decision	Why
Front end	React 18 + Vite + TypeScript, built as an installable PWA	One codebase for phone and desktop; the design is mobile first; installs from Safari and Chrome without an app store
Hosting	Cloudflare Pages, with a preview URL per branch	Free tier, global CDN, no server to run; Pages is only static files, so the backend lives elsewhere
Database	Supabase Postgres, hosted	Relational data (restaurants, equipment, tickets, service records); row-level security enforces roles
Auth	Supabase Auth: email + password, invite-only, TOTP 2-step	Invite flow and MFA built in; the design's sign-in, reset, 2-step and invite screens map onto it directly

Area	Decision	Why
Data access	The React app reads and writes Supabase over its client library; every table has RLS policies	Fewer moving parts than an API layer; permissions live in one place and are tested as SQL
Server logic	Supabase Edge Functions (Deno, TypeScript) for invites, push, imports, QR label PDFs, scheduled jobs	Python was the first thought but is not required; one language across the whole app, and Edge Functions deploy with the Supabase CLI
Files	Supabase Storage, photos resized to 1600 px in the browser before upload	Same permission rules as the tables; invoices and photos stay small
Email	Resend, through Supabase's custom SMTP setting plus a sending Edge Function	Supabase's built-in email is rate limited to a few per hour
Push	Web Push (VAPID keys), subscriptions stored per device, sent from an Edge Function	Push first, as asked; iPhone needs the app installed to the home screen, so email is the fallback for anyone who has not installed
UI	Tailwind with the design tokens as CSS variables, headless Radix primitives, components built to match the design	The design is custom, so a styled library would fight it; tokens give light and dark mode from day one
Data layer	TanStack Query over the Supabase client; React Router	Caching, retries and optimistic updates without a store to maintain
Testing	Vitest for logic, pgTAP for RLS policies, Playwright on Chromium and WebKit for flows	WebKit in Playwright is the stand-in for an iPhone until a real-device pass
Repo and CI	One GitHub repo (`app/`, `supabase/`), GitHub Actions: typecheck, tests, Cloudflare Pages deploy, Supabase migrations on merge to main	Solo, AI-assisted build: every change goes through the same gate
Offline	Later (Phase 8). v1 caches what you have read; writes need a connection	The queued-ticket flow in the design is real work on its own
Permissions	Four roles with defaults from the Roles & rights screen, overridable per person	Managers can be granted equipment edits and cost visibility by an admin; the database checks the grant, not the screen

Two things stay open until the first demo: the production domain (QR labels bake the URL in, so it should be settled before labels are printed) and whether the group wants Google or Microsoft sign-on later.

Architecture



The phone never holds a secret: it signs in with Supabase Auth, then reads and writes tables and files under row-level security. Edge Functions run with the service key for the few jobs a phone cannot do safely (send an invite, push to other people's devices, process an import, render a label sheet, create next month's walkthroughs on a schedule). GitHub Actions deploys the app to Cloudflare Pages and the migrations and functions to Supabase on every merge.

Data model

Twenty tables in five groups. Tickets are the center; equipment, service records and walkthroughs all hang off them. The nine-table model in the earlier doc grew by the walkthrough, contractor and rights tables the design added.

Group	Tables	Notes
Org and access	`restaurants`, `profiles`, `memberships`, `invites`, `rights`, `push_subscriptions`, `audit_log`	`restaurants` holds name, address, phone, accent color, manager. `memberships` = one row per person per restaurant with a role (manager, ops, leadership, admin); ops and leadership default to all restaurants. `rights` = the Roles & rights grid: a default row per role, plus per-person override rows (can_edit_equipment, can_see_costs, can_set_walkthrough_schedule, …)
Equipment	`equipment`, `categories`	Category carries area (FOH or BOH). Equipment has brand, model, serial, installed, warranty, location note, manual file, retired_at
Tickets	`tickets`, `ticket_updates`, `ticket_photos`	Ticket: restaurant, equipment (optional), category, title, description, urgency, status, assignee, source (reported or walkthrough), walkthrough_id, reopened_at. Status is an enum: open, in_progress, waiting_parts, scheduled, reopened, closed. Updates carry text, photo, internal flag, and system events (assigned, status change, closed, reopened)
Service and money	`contractors`, `service_records`, `service_line_items`, `invoices`	A service record belongs to a ticket and optionally a contractor; line items are labor, parts, trip. An invoice is a file plus number, amount and status (attached, missing). Per-unit and per-contractor spend are views over service records
Walkthroughs	`walkthrough_schedules`, `walkthrough_templates`, `walkthrough_items`, `walkthroughs`, `walkthrough_checks`, `inhouse_fixes`, `quarterly_tasks`	A schedule is group-wide or per restaurant (rotation of months). A walkthrough is one restaurant for one month; checks are its item results (good, in-house, ticket). In-house fixes link back to the check and can convert to a ticket. Quarterly tasks are group-wide with per-restaurant overrides and removals

How rights work. The JWT carries only the user id. A SQL function `can(action, restaurant_id)` reads `memberships` and `rights` and is used by every policy, so a manager with `can_edit_equipment` granted by an admin passes the same check an ops manager does. Leadership policies are select-only. The service role used by Edge Functions bypasses RLS, which is why those functions check the caller themselves before acting.

Counting and costs (open tickets per restaurant, spend this year per unit, visits per contractor) are Postgres views, not client-side sums, so every screen shows the same numbers.

The first migration should land all five groups even though phases 1 to 4 use only the first three; adding walkthrough tables later would mean reworking the ticket table's `source` and `walkthrough_id` columns.

Phase 0 · Foundation

An empty app that installs on a phone, switches between light and dark, deploys on every merge, and has a database whose permissions are already tested. Nothing a user would call a feature, but every later phase moves faster for it.

Ships

- Repo with `app/` (Vite + React + TypeScript) and `supabase/` (migrations, functions, seed, tests); GitHub Actions running typecheck, Vitest, pgTAP and Playwright on Chromium and WebKit
- Cloudflare Pages connected to the repo: production on `main`, a preview URL per pull request
- Supabase project with the full schema from the data model, RLS on every table, the `can()` function, and pgTAP tests that sign in as each role and assert what it can and cannot read
- Seed script with the four restaurants, their colors, sample users for each role, equipment and tickets matching the design's sample data, so every demo starts from the same state
- Design tokens as CSS variables (colors, spacing, type, radii) with a dark set; Tailwind wired to them; the `prefers-color-scheme` default plus a manual toggle saved per device
- Component kit built from the design: button, pill, card, list row, bottom tab bar, sheet, field, select, switch, segmented control, avatar, toast, skeleton. Each one has a Storybook-style page in the app under `/kit` for checking both themes
- PWA shell: manifest, icons, splash, service worker with precaching, install prompt, `100dvh` and safe-area padding, 16 px minimum input size
- App frame: routing, auth guard, the four role shells with their tab bars, loading and error boundaries

Screens from the design: Loading states (spinner, list and detail skeletons), Error 404, 500, no access

Done when: a teammate can open the preview URL on an iPhone, add it to the home screen, toggle dark mode, and the RLS tests pass for all four roles.

Phase 1 · Accounts and access

Real people can be invited, sign in, and land in the right role, with rights an admin can adjust. Everything after this is gated on who is looking, so it comes first.

Ships

- Admin: Restaurants list and Add restaurant (name, address, phone, manager, accent color with the preset swatches and custom picker, taken-color check)
- Admin: Users list with filters (all, pending, managers, ops), Invite user (name, email, role, restaurants), resend and revoke, user profile with per-person rights override, View as (read-only impersonation, logged to the audit table)
- Admin: Roles & rights grid, editable defaults with the confirm sheet, locked admin column
- Invite email through Resend; invite link single-use, 72-hour expiry; Set up account screen (name, password rules, confirm)
- Sign in, password reset (request, sent, email, new password, done), expired link and signed-out screens
- 2-step verification: TOTP with authenticator app (choose, scan, verify, backup codes) required for admins and offered to everyone; the SMS option waits unless the group asks for it, since it adds a Twilio cost
- Profile (name, phone, restaurant, theme toggle, sign out) and Help & support (report a problem to a support inbox, FAQ)
- Edge Functions: `invite-user`, `revoke-user`, `send-email`

Screens from the design: Admin · Users, Invite user, Roles & rights, User profile, View as, Restaurants, Add restaurant; Invite email, Set up account, Sign in, Password reset 1 to 5, 2-step setup 1 to 4 and the sign-in code screen, Profile, Help & support, Error · link expired, no access, signed out

Done when: an admin invites a manager from a phone, the manager sets up the account from the email on another phone, and the RLS tests show the manager sees only their restaurant. Granting `can_see_costs` to that manager in Roles & rights changes what the database returns, with no app change.

Phase 2 · Tickets, the manager loop

A restaurant manager can report a problem from a phone in under a minute and follow it to closed. This is the first thing worth showing the Four Corners team, even before Ops has its own screens.

Ships

- Open tickets list: Tickets and In-house tabs with counts, search, All / Front / Back segments, the restaurant label with its color square, cards with status pills and icons
- Create ticket in two steps: area and category grid (counts per category), then the form (title, description, photos, urgency, optional equipment picker filtered to that restaurant)
- Ticket detail: header with status and urgency, description, equipment card, the update thread with photos, the composer with camera, system events in the timeline
- Edit or close ticket (manager side), with the closed state read-only
- Closed ticket: resolution, contractor and cost shown only when `can_see_costs` is granted, 30-day reopen window, the Reopen ticket sheet that sets status to Reopened and notifies Ops

- Photo upload: resized to 1600 px in the browser, stored under the ticket, thumbnails in the thread
- Empty and no-results states for the lists; the save-failed validation state on the form
- Status icons, pills and the restaurant color square as shared components, since every later list reuses them

Screens from the design: Manager · Open tickets, Open in-house tasks, Create ticket area, Create ticket form, Open ticket, Edit / close ticket, Closed ticket; Empty · Manager open tickets; No results · ticket search; Error · validation on save

Done when: a manager at Federale's Logan Square creates a ticket with a photo on an iPhone, it appears in the database with the right restaurant, and a second manager at Porter cannot see it. Ops can still only see tickets through the database at this point; that is Phase 3.

Phase 3 · Ops dashboard and ticket work

The ops managers can run their day in the app: see every restaurant's open work, pick up tickets, talk to the restaurant, close, and handle a reopen. With push, this is the point where the app replaces texting.

Ships

- Ops dashboard: Open and Closed tabs, All / Mine segments, the filter sheet (status, urgent, walkthrough only, area, category, sort) with chips, cards with restaurant color squares, walkthrough tags and assignee initials; the restaurant picker (All restaurants plus each one) that scopes every Ops screen
- Ops ticket detail: status and assignee dropdowns, description, equipment card, service and cost section (empty until Phase 5), updates thread with the internal-note toggle ("Visible to restaurant"), composer
- Closed tab grouped by month with totals; closed ticket detail
- Reopened ticket: the Start work sheet (note, assigns to you, In progress) and the Close again sheet (required reply to the manager, back to Closed)
- Push notifications: subscribe on install, store per device, Edge Function `send-push` fired by database triggers for new ticket, assignment, status change, new update visible to the restaurant, reopen. Email fallback for anyone with no subscription. A Notifications section on Profile to turn each kind on or off
- Empty states for the dashboard and Closed tab; loading skeleton for the card feed

Screens from the design: Ops · Dashboard, Ticket detail, Closed tab, Reopened ticket; Empty · Ops dashboard, Ops closed tab; Loading · card feed skeleton; Error · something went wrong

Done when: a manager creates a ticket and the ops manager's phone buzzes; the ops manager assigns it, posts an update the manager sees, closes it with a comment; the manager reopens it and Ops sees Reopened on the dashboard. Playwright runs that whole loop on WebKit.

Phase 4 · Equipment and QR labels · first demo

Every unit has a page, a QR label, and a history. This closes the loop that EcoTrak does not have, and it is the milestone to put on the ops team's phones: phases 1 to 4 together are a complete ticketing app with equipment.

Ships

- Ops equipment list: search, filter sheet (restaurant, area, category), category disclosures with open-ticket badges, rows with restaurant color, brand, serial and this year's spend
- Manager equipment list: same list scoped to their restaurant, with Add equipment only when `can_edit_equipment` is granted
- Equipment detail tabs: Details (serial, QR, brand, model, category, location, installed, warranty, manual), Tickets (open on top, closed in a disclosure), Service & cost (lifetime and 2026 cost, visits, contractors on this unit, service log). Cost tabs hidden without `can_see_costs`
- Add and Edit equipment, Retire equipment; the saved screen with the QR label and Print
- QR: each label encodes `https://<domain>/e/<id>`; the phone's camera opens the unit; a signed-out scan goes to sign-in and back. Label sheets as a PDF from an Edge Function `qr-labels` (per restaurant or selected units), sized for common label stock
- Create ticket from a unit, with the unit prefilled; the equipment breadcrumb with the restaurant color on every equipment screen
- Empty states: no equipment, no tickets on a unit, no service yet, QR not recognized; equipment loading skeletons

Screens from the design: Ops · Equipment list, Add equipment, Edit equipment, Equipment added + QR label, Equipment details, Equipment tickets, Equipment service history; Manager · Equipment, Equipment details and tickets tabs; Loading · QR scan, equipment list and detail; Empty · manager equipment, equipment tickets, new equipment; Error · QR code not recognized

Demo milestone. Seed the four restaurants with their real equipment where it is known, print a label sheet for one restaurant, hand the assistant ops manager and one restaurant manager an install link, and let them run a real ticket for a week. The next-steps slide in the deck becomes the agenda for that conversation.

Done when: scanning a printed label on an iPhone opens the unit in the installed app, and a ticket created from it shows up under that unit's Tickets tab and on the Ops dashboard.

Phase 5 · Contractors, service records and invoices

Every repair gets a cost, a contractor and an invoice, and the spend views on equipment, contractors and the dashboard start showing real numbers. This is the section of the deck the head of ops is paying for.

Ships

- Log service from a ticket: in-house or contractor, contractor select with "Add new", technician, invoice number, service date, labor, parts, trip fee with a live total, work notes, attach invoice; the inline saving state
- Service record: invoice card (view, download, replace, missing state), cost breakdown, details; Edit service record with delete
- Contractor directory: search, trade chips, sort by spend or last used, preferred stars, insurance-expiring flags, one-tap call and email
- Contractor detail: spend, visits, last visit, details, Visits and Invoices tabs; Add and Edit contractor with the preferred switch
- Invoices tab: by month, missing-invoice filter and count, sort, rows linking to service records
- Spend views in Postgres feeding the ticket total, unit lifetime and yearly cost, contractor spend and visits, and the Closed tab month totals
- Empty states: contractor directory, contractor search; the Add contractor link from Log service

Screens from the design: Ops · Log service & cost, Service record, Edit service record, Contractor directory, Contractor detail, Add contractor, Edit contractor, Invoices; Loading · inline button spinner; Empty · contractor directory; No results · contractor search

Done when: logging a $397.40 Coldline visit on a ticket raises that unit's 2026 cost, Coldline's spend, and the Closed tab's month total by the same amount, with the invoice PDF openable from all three places.

Phase 6 · Monthly walkthroughs

Managers walk a checklist each month, log fixes and tickets as they go, and Ops sees who is on track. It is the biggest phase, so it is split into two halves that each ship on their own.

6a · The manager's walkthrough

- Walkthrough home for the month: progress by area, due date, what is flagged; the all-done state
- Checklist by area (BOH, FOH, quarterly tasks) with the per-item Good / In-house / Ticket controls, group disclosures, the help panel
- Quick ticket sheet from an item (creates a ticket with `source = walkthrough` and the Walkthrough tag), and the walkthrough edit/close ticket screen
- In-house fixes: the shared list opened filtered from an item or whole from review, check off, delete with confirm
- Review & submit with the partial-submit warning; the all-tickets list from review
- Walkthrough creation: `pg_cron` opens next month's walkthrough per restaurant from its schedule; reminders 7 and 2 days before due go out by push and email

6b · Ops and the schedule

- Ops walkthroughs overview: progress per restaurant, submitted dates, the last-6-months list; one restaurant's walkthrough with flagged items, the reminder button, past walkthroughs
- Ops in-house fixes view, read-only with Convert to ticket
- Walkthrough schedule: group rotation, per-restaurant own schedule and switch back, the 12-month preview, settings
- Quarterly tasks: group list with per-restaurant overrides, removals and restores, the task editor with directions and video link
- Checklist template editor so Ops can change the items without a deploy (the design has no screen for this yet; a simple list editor is enough)

Screens from the design: Walkthrough · Manager (11 screens: this month, checklist, quick ticket, quarterly task, all done, review, partial warning, in-house fixes from item and from review, edit ticket, all tickets); Ops (5 screens: all restaurants, one restaurant, schedule, quarterly tasks, in-house fixes)

Done when: October's walkthrough is created automatically for all four restaurants, a manager submits one from a phone with two tickets and one fix, those tickets show the Walkthrough tag on the Ops dashboard, and the Ops overview shows one of four submitted.

Phase 7 · Leadership, Admin tickets and imports

The read-only views for leadership and admin, and the spreadsheet imports that make a real rollout to all nine restaurants practical.

Ships

- Leadership: Overview (open, urgent, spend, time to close, by restaurant), Restaurant view with open tickets and equipment, Ticket detail, Contractors directory and invoices, Contractor detail, Service record, Walkthroughs; all read-only, driven by the same views as Ops
- Admin: Tickets list and ticket detail, read-only with the view-only banner
- Import contractors and Import equipment: choose a file (xlsx or csv), match columns with guesses and one-tap suggestions, review rows with grouped fixes and duplicate handling (skip or update), done screen with counts, QR labels for imported units, 24-hour undo. Parsing and matching run in an Edge Function `import` so the phone only uploads the file
- Import template downloads for both lists
- Entry points: the banners on Add contractor and Add equipment, the button on the empty contractor list

Screens from the design: Leadership (9 screens plus 2 empty states and the loading skeleton); Admin · Tickets, Ticket detail; Ops · Import contractors 1 to 4, Import equipment 1 to 4

Done when: the assistant ops manager imports the group's real contractor spreadsheet from a phone with no help, and leadership can open the Overview on a laptop and see the same totals Ops sees.

Phase 8 · Desktop layouts, offline, and later

These come after the group is using the app daily, in whatever order the feedback puts them.

- **Desktop layouts.** Tablet and desktop up to 1440 px: the tab bar becomes a sidebar, lists become list-plus-detail panes, sheets become dialogs, and the import review becomes a table. The phone layouts stay as the base; this is CSS and a few layout components, not new screens
- **Offline.** Cache the lists and tickets you have opened; queue new tickets and updates with the Not sent yet card and the Saved-at banner; retry on reconnect; handle a conflict by appending, never overwriting. This is the Error · Offline screen from the design
- **Reports.** Monthly spend per restaurant and per category as a PDF or CSV for leadership; repeat-repair flags on a unit
- **Single sign-on** with Google Workspace or Microsoft 365 if the group wants it; Supabase Auth supports both as providers
- **App Store wrapper** with Capacitor only if someone asks for it; the PWA covers the need
- **Second organization.** The schema has one group today; a `tenant_id` on restaurants and the `can()` function would let the same deployment serve another restaurant group

Working method

One person with AI help can move fast on this stack as long as every phase has the same shape: schema and policies first, then the screens against the design, then a Playwright flow that proves the done test. Order inside a phase:

1. **Migration and policies.** Write the tables, the `can()` cases and the pgTAP tests for the phase before any UI. Tests run against a local Supabase (`supabase start`), so the database is right before a screen exists
2. **Data hooks.** One TanStack Query hook per screen's data, typed from the generated Supabase types (`supabase gen types`), with the seed data as fixtures
3. **Screens against the design.** Build each screen from its board on the design canvas, in both themes, in the `/kit` page first when a new component is needed. The canvas's sample data and the seed data match on purpose, so a screen is done when it looks like its board
4. **Flow test.** One Playwright test per phase on Chromium and WebKit, driving the done test end to end against the seeded local stack
5. **Preview and try on a phone.** Every pull request gets a Cloudflare preview URL; open it on an iPhone before merging. A BrowserStack real-device pass before each milestone handed to the client

Conventions

- Small pull requests, one screen or one function each, merged to `main` only when CI is green; `main` deploys to production and applies migrations automatically

- A `CLAUDE.md` in the repo holding the stack, the folder layout, the token names, the done test for the current phase and the rules from the earlier doc (16 px inputs, `100dvh`, safe areas), so each AI session starts with the same context
- No restaurant data in the repo; the seed is fictional except restaurant names and colors
- Secrets only in Supabase and GitHub Actions settings, never in `.env` files that get committed
- Each phase ends with a short written note of what changed and what the client should try, which becomes the message to the ops team

What done means for a phase: its done test passes in CI on both engines, the screens match their boards in light and dark, RLS tests cover its tables, and it has been tried on a real phone from the preview URL.

Design reference

The agents building each phase cannot open the design canvas or the slide deck, so the design is exported into the repo as a `design/` folder (the `ontrack-design.zip` from October 28). It is the source of truth for how every screen looks; the canvas stays the place where the design changes, and the folder is re-exported whole when it does.

File	What it holds	When an agent reads it
`design/INDEX.md`	Every screen: file name, title, role, and the phase it belongs to	At the start of a phase, to list the screens to build
`design/screens/<Name>.dc.html`	The screen's markup, CSS, copy and sample data as drawn on the canvas, with its `data-props` states and the `href` links that define navigation	Before building or changing that screen
`design/screens/<Name>.png`	The same screen rendered at 390×844, light mode, 2x	Alongside the HTML, as the picture to match
`design/tokens.md`	Colors, type scale, status pill colors and icons, restaurant colors and presets, radii, spacing, and the proposed dark theme	When setting up Tailwind and the CSS variables in Phase 0, and whenever a value is in doubt
`design/canvas.json`	Board positions and section titles from the canvas	Rarely; it shows how screens group on the board
`design/README.md`	How to use the folder and the rule to paste into `CLAUDE.md`	Once, when the repo is set up

The rule that goes in the repo's `CLAUDE.md`, so every agent session follows it: before building or changing any screen, read its file in `design/screens/` and look at its PNG; match it, do not reinterpret

it; styles come only from `design/tokens.md`; a phase is done when every screen listed for it in `design/INDEX.md` renders on a phone and matches its PNG, in light and dark mode.

Two things the export does not carry. The screens are light mode only, so dark mode is built from the proposed values in `tokens.md` and checked in `/kit` before any screen uses it. And the HTML files depend on the canvas runtime, so they are read, not run; the PNG is the rendered view, and a new state that needs a picture gets a fresh export rather than a hand edit.

The slide deck is for people, not agents. If a screen in the deck and the same screen in `design/` ever disagree, `design/` wins, and the deck gets updated from the canvas.

Risks and open items

Risk	Why it matters	What to do
iPhone push only works when the app is installed to the home screen	The ops team may open it in Safari and wonder why nothing buzzes	Install prompt on first sign-in with a two-step picture; email fallback until a subscription exists; a Profile row that shows whether push is on
RLS policies get complex with per-person rights overrides	A bug here leaks another restaurant's data	Every rule goes through one `can()` function with pgTAP tests per role; never a second code path in the app
Supabase free tier pauses after a week idle	A paused project during the pilot looks like a crash	Move to Pro ($25 a month) before handing the app to the client
QR labels bake in the domain	Reprinting labels is a real cost once they are on units	Buy the domain before Phase 4; use a short path like `/e/` so labels stay small
Resend domain needs DNS records before invite emails land in inboxes	First invites bounce or go to spam	Set up SPF, DKIM and DMARC on the sending domain in Phase 1, send a test to a Gmail and an Outlook address
The walkthrough phase is large	It can stall the pilot	It is after the demo milestone and split in two; 6a alone is useful
Solo build with AI help drifts from the design	Screens that look close but not right erode trust with a team that already struggled with the canvas	Build from the boards one at a time, check both themes in `/kit`, and put the preview on a phone before merging

Open items

- [ ] Production domain, so QR labels and emails have a stable base
- [ ] Whether the group uses Google Workspace or Microsoft 365, for sign-on later
- [ ] The real walkthrough checklist items per area (the design has 48 placeholders)

- [ ] Who at Four Corners is the first admin, and whether the head of ops wants admin or ops
- [ ] Label stock size for the QR sheets
- [ ] Whether SMS 2-step is wanted, given its per-message cost